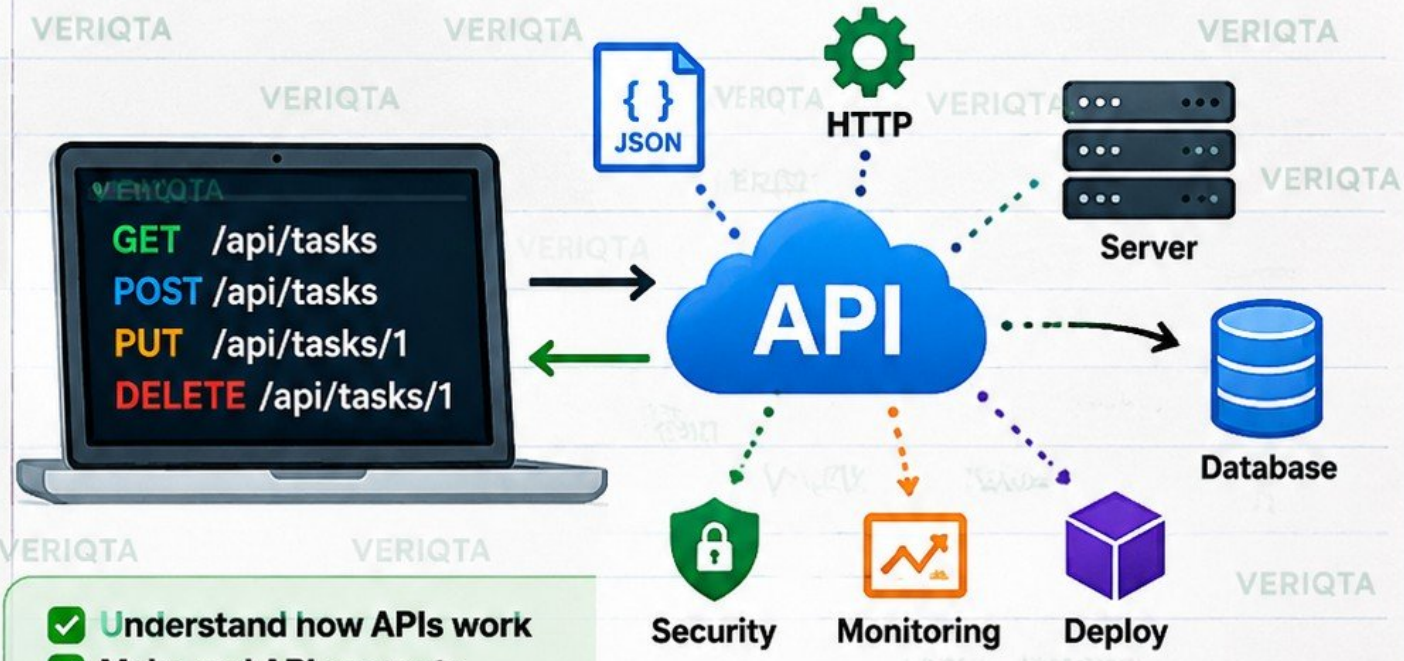
VERIQTA

API Handbook for DevOps Beginners



- ✅ Understand how APIs work
- ✅ Make real API requests
- ✅ Work with JSON data
- ✅ Secure and troubleshoot APIs
- ✅ Use APIs in real DevOps environments
- ✅ Build a strong foundation for production

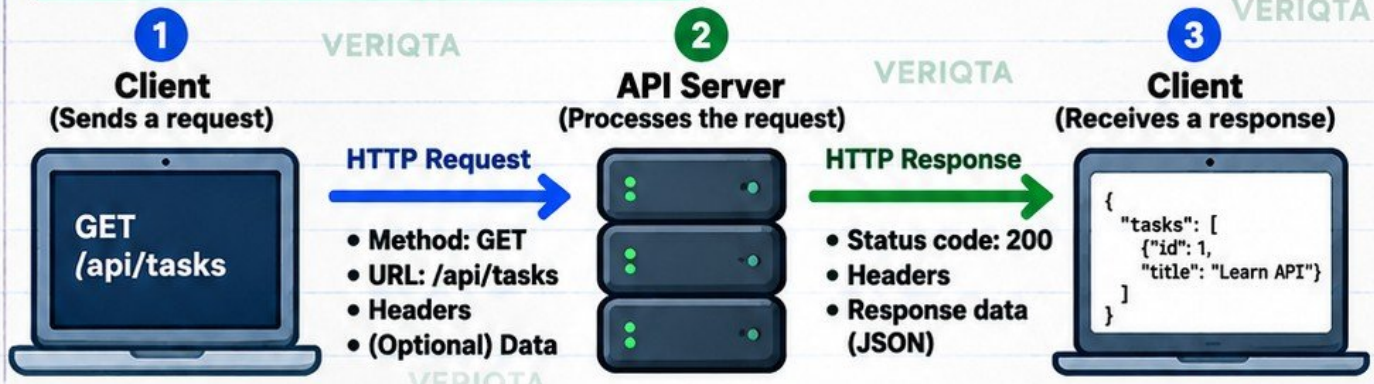
@veriqta Instagram | @veriqta GitHub | @veriqta LinkedIn | @veriqta Telegram | @veriqta X | @veriqta Discord

What Actually Happens When You Call an API?

1. What is an API?

- An API (Application Programming Interface) is a set of rules that allows different applications to communicate.
- It defines how a client can request data or perform an action and how the server responds.
- An API is a communication interface, not necessarily a separate application or server.

2. How an API Request Works



3. Step-by-Step Process

1. **The client sends an HTTP request**
   The request includes a method (GET, POST, etc), a URL, headers and sometimes data.
2. **The server receives the request**
   The API server checks the request, validates it, and performs the required action.
3. **The server processes the request**
   It may read data from a database, run business logic, or interact with other services.
4. **The server sends a response**
   The response includes a status code, headers, and data (usually in JSON).
5. **The client receives and uses the response**
   The client reads the data and takes the next action (e.g. display it in an app).

4. Real Example

Request from a client:

```
GET /api/tasks HTTP/1.1
Host: example.com
Accept: application/json
```

Response from the server:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "tasks": [
    {"id": 1, "title": "Learn API",
     "status": "open"}
  ]
}
```

Engineering Insight

An API is a communication interface, not necessarily a separate application or server. An API can be a small part of a larger application, and many different clients can use the same API.

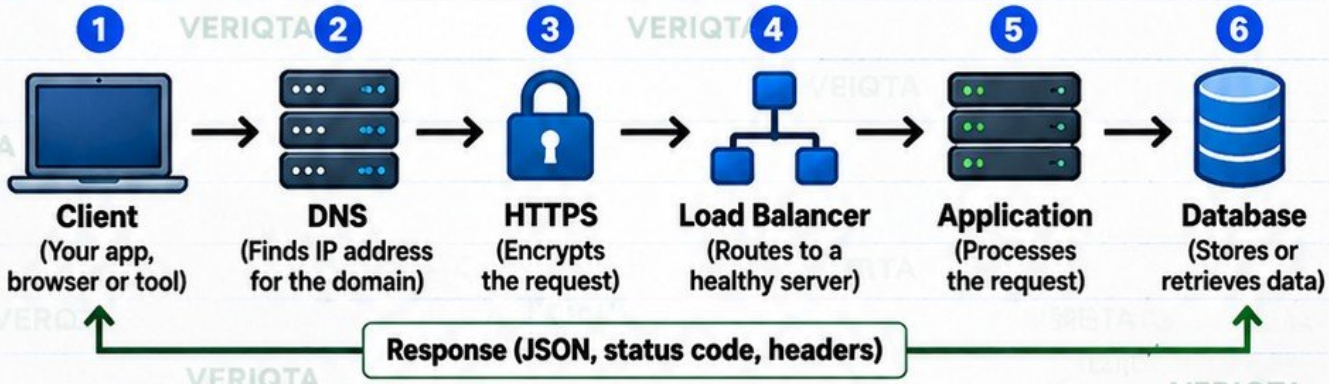
Common Beginner Mistake

Thinking an API is only a website or a separate server. In reality, an API can be part of a web application, a microservice, or even a cloud service.

The Journey of an API Request

When you call an API, your request travels through multiple components before it reaches the application and gets a response.

1. Simplified Production Request Path



2. What Each Component Does

1 Client

- Sends an HTTP request to a domain (e.g. api.example.com).
- Includes method, URL, headers and optional data.
- Can be a browser, mobile app, backend service or a tool like curl.

2 DNS (Domain Name System)

- Converts the domain name (e.g. api.example.com) to an IP address (e.g. 203.0.113.10).
- Allows your device to find where the API is hosted.
- Usually handled automatically by your operating system.

3 HTTPS (TLS Encryption)

- Creates a secure, encrypted connection between your client and the server.
- Protects the data from being read or changed by others.
- Uses a digital certificate to verify the server's identity.

4 Load Balancer

- Receives the request and routes it to a healthy application server.
- Distributes traffic across multiple servers.
- Can also perform health checks, SSL termination and traffic controls.

5 Application (API Server)

- Receives the request from the load balancer.
- Validates the request, checks authentication and permissions.
- Runs the application logic (e.g. get or create a task).
- Communicates with a database or other services if needed.

6 Database

- Stores or retrieves the data requested by the application.
- Can be a relational database (e.g. PostgreSQL, MySQL) or a NoSQL database (e.g. MongoDB).
- Sends the data back to the application, which then returns a response to the client.

Failure Insight

- A failed API request does not automatically mean the application code is broken.
- The issue could be with DNS, network connectivity, SSL/TLS, the load balancer, a database, or another downstream service.

Common Issues at Each Step

- DNS: domain not found or wrong IP address.
- HTTPS: certificate errors or connection timeout.
- Load balancer: unhealthy targets or misconfiguration.
- Application: authentication failure or internal error.
- Database: slow queries or connection issues.
- Network: firewalls blocking the request.

Anatomy of an HTTP Request

An HTTP request is a message a client sends to a server. It tells the server what action to perform and can include data, headers and parameters.

1. Example Request

```
GET /api/tasks?status=open&priority=high HTTP/1.1
```

Method (GET)

- Tells the server what action to perform.
- Common methods: GET, POST, PUT, PATCH, DELETE.

Path (/api/tasks)

- Identifies the resource on the server.
- In this example, it refers to the tasks resource.

Query Parameters (?status=open&priority=high)

- Optional key-value pairs added to the URL.
- Used to filter, search or modify the request.
- Multiple parameters are separated with &.

HTTP Version (HTTP/1.1)

- Specifies the HTTP protocol version.
- Common versions are HTTP/1.1 and HTTP/2.

2. Full HTTP Request Example (with headers)

```
GET /api/tasks?status=open HTTP/1.1
Host: api.taskflow.com
Accept: application/json
Authorization: Bearer eyJhbGciOi... (token)
User-Agent: curl/8.0.1
Accept-Encoding: gzip, deflate
Connection: keep-alive
```

Headers

- Provide additional information about the request.
- Common headers:
  - **Host:** the domain name
  - **Accept:** the response format (e.g. JSON)
  - **Authorization:** authentication token
  - **User-Agent:** identifies the client

3. Optional Request Body (used with POST, PUT, PATCH)

```
POST /api/tasks HTTP/1.1
Host: api.taskflow.com
Content-Type: application/json
Authorication: Bearer eyJhbGciOi...
{
   "title": "Learn DevOps",
   "status": "open",
   "priority": "high"
}
```

Request Body

- Contains the data you want to send to the server.
- Commonly used with POST, PUT and PATCH.
- Usually in JSON format.
- In this example, we are creating a new task.

Common Beginner Mistake

Confusing query parameters with data sent in the request body. Query parameters are part of the URL and are used for filtering or searching. The request body is used to send data to create or update resources, usually with POST, PUT or PATCH.

Key Takeaway

An HTTP request is made up of a method, URL (path and optional query parameters), headers and an optional body. Each part has a specific purpose and helps the server understand what you want to do.

HTTP Methods and Their Real Behavior

VERIQTA

HTTP methods tell the server what action to perform on a resource. Here we use a TaskFlow API to see how each method works in real scenarios.

Method	Action on TaskFlow Tasks	Example Request	Expected Behavior	Safe?	Idempotent?
GET	Retrieve a list of tasks or a single task	`GET /api/tasks` `GET /api/tasks/1`	Returns task data without changing anything.	Yes	Yes
POST	Create a new task	`POST /api/tasks` `{ "title": "Learn API", "status": "open" }`	Creates a new task and returns the created resource.	No	No
PUT	Replace an existing task completely	`PUT /api/tasks/1` `{ "title": "Learn DevOps", "status": "in_progress" }`	Replaces the entire task with the provided data.	No	Yes
PATCH	Update specific fields of a task	`PATCH /api/tasks/1` `{ "status": "done" }`	Updates only the specified fields (e.g. status).	No	Yes
DELETE	Delete a task	`DELETE /api/tasks/1`	Removes the task from the system.	No	Yes

Safe and Idempotent Methods

- **Safe** means the request does not change data on the server.
- **Idempotent** means you can send the same request multiple times and get the same result.
- **GET** is both **safe** and idempotent.
- **PUT** and **DELETE** are idempotent but not safe (because they change data).
- **POST** is not safe and not idempotent (because it usually creates a new resource).

Production Insight

Retrying POST requests carelessly can create duplicate resources.

- If a network timeout happens, you might send the same POST request again.
- The server may create the task twice.
- Use idempotency keys when available or design your API to prevent duplicates.

Common Beginner Mistakes

- Using POST when you should use PUT or PATCH.
- Sending a request with the wrong method.
- Thinking GET can create or update data (it cannot).
- Not understanding that POST requests can create duplicates.
- Confusing safe and idempotent operations.

Key Takeaway

Choose the correct HTTP method based on the action you want to perform. Understand which methods are safe and idempotent to avoid unexpected changes or duplicate data.

HTTP Status Codes: Read the Evidence

VERIQTA

HTTP status codes tell you the result of an API request. They help you quickly understand what happened and where to start troubleshooting.

2xx - Success

- The request succeeded.
- The server performed the requested action.
- Examples: 200, 201, 204

3xx - Redirection

- The client needs to take another action.
- Usually redirects to a different URL.
- Examples: 301, 302

4xx - Client Error

- The request is invalid or not allowed.
- The client needs to fix the request.
- Examples: 400, 401, 403, 404, 429

5xx - Server Error

- The server failed to process the request.
- Usually a server or upstream service issue.
- Examples: 500, 502, 503, 504

Status Code	Meaning	When It Happens	First Diagnostic Question
200	OK	The request succeeded and returns data.	Did I get the expected data?
201	Created	A new resource was created (e.g. a new task).	Was the resource created correctly?
204	No Content	The request succeeded but returns no body.	Is no data expected for this request?
400	Bad Request	The request is invalid (e.g. malformed JSON).	Is my request format correct (URL, headers, JSON, parameters)?
401	Unauthorized	Authentication is missing or invalid.	Did I provide a valid token or API key?
403	Forbidden	You are authenticated but do not have permission.	Does my account have the required permissions?
404	Not Found	The requested resource does not exist.	Is the URL correct? Does the resource ID exist?
429	Too Many Requests	You have exceeded the rate limit.	Am I sending too many requests? Should I wait and retry later?
500	Internal Server Error	A general server error occurred.	Is there an issue in the application logs?
502	Bad Gateway	An intermediary (e.g. load balancer or proxy) received an invalid response from an upstream service.	Is the upstream service running? Is there a network or load balancer issue?
503	Service Unavailable	The server is temporarily unavailable (maintenance, overload or unhealthy).	Is the service under maintenance? Are the service instances healthy?
504	Gateway Timeout	The server did not receive a response from an upstream service in time.	Is the upstream service slow or unreachable? Is there a network timeout?

Troubleshooting Insight

A 502 response often points toward an intermediary's communication with an upstream service, not necessarily a bug in the client.

Common Beginner Mistakes

- Assuming every error is a problem with your code.
- Ignoring the status code and only looking at the message.
- Not checking headers and response body.
- Retrying requests without understanding the cause.

JSON: The Data Inside API Responses

JSON (JavaScript Object Notation) is a lightweight data format used by APIs to send and receive data. It is easy to read and widely used in web applications.

1. JSON Data Types

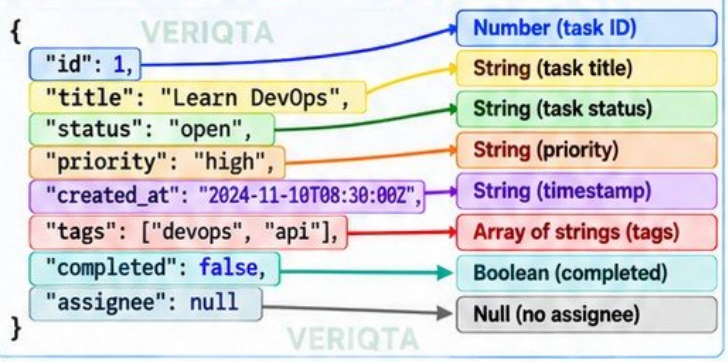
Object	A collection of key-value pairs (enclosed in { }).	Example: { "id": 1, "title": "Learn" }
Array	A list of values (enclosed in [ ]).	Example: [1, 2, 3]
String	Text data (enclosed in quotes).	Example: "Learn DevOps"
Number	Numeric data (integer or decimal).	Example: 1 or 3.14
Boolean	True or false value.	Example: true or false
null	Represents no value.	Example: null

2. Real TaskFlow API Response

Here is a realistic JSON response from a TaskFlow API that returns a list of tasks.

```
{
  "tasks": [
    {
      "id": 1,
      "title": "Learn DevOps",
      "status": "open",
      "priority": "high",
      "created_at": "2024-11-10T08:30:00Z",
      "due_date": "2024-11-20T00:00:00Z",
      "tags": ["devops", "api"],
      "completed": false,
      "assignee": null
    },
    {
      "id": 2,
      "title": "Build CI/CD Pipeline",
      "status": "in_progress",
      "priority": "medium",
      "created_at": "2024-11-11T14:20:00Z",
      "due_date": null,
      "tags": ["github", "actions"],
      "completed": false,
      "assignee": "ann"
    }
  ]
}
```

3. Understanding the JSON Structure



4. Practical: Find Key Information

From the first task in the response above, identify:

- **Task ID:** 1
- **Title:** Learn DevOps
- **Status:** open
- **Timestamp:** 2024-11-10T08:30:00Z

Common Beginner Mistake

- Assuming every API response has the same JSON structure.
- Some APIs return a single object, others return an array.
- The field names, nesting and data types can be different.
- Always check the actual response format before writing scripts or integrations.

Key Takeaway

- JSON is used to structure data in API responses.
- It can contain objects, arrays, strings, numbers, booleans and null values.
- Always check the actual response before processing the data.

Real-World Tip

- Use tools like jq (Linux/Mac) or online JSON viewers to read and format JSON responses.
- In real APIs, additional fields (like pagination, metadata or error messages) are often included.

Your First API Request With curl

curl is a command-line tool used to make HTTP requests to APIs. It allows you to send requests, view responses, and test APIs directly from your terminal.

1. Basic curl Command

The simplest way to call an API is to provide the URL.

```
curl https://httpbin.org/get
```

2. Understanding the URL

The URL tells curl where to send the request.



3. Useful curl Options

These options help you see more information in the response.

Option	What it Does
-i	Shews the response headers along with the body.
-s	Silent mode. Hides the progress meter for a cleaner output.
-S	Shows an error message if the request fails (useful with -s).
-sS	Silent mode but still shows errors. Commonly used in scripts.

4. Practice: Make Your First API Request

Run this command in your terminal:

```
curl -i https://httpbin.org/get
```

5. Example Output (Truncated)

You will see response headers followed by a JSON body.

```
HTTP/1.1 200 OK
Date: Sat, 15 Nov 2025 12:00:00 GMT
Content-Type: application/json
Content-Length: 554
Server: gunicorn
Access-Control-Allow-Origin: *

{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Host": "httpbin.org",
    "User-Agent": "curl/8.0.1"
  },
  "origin": "102.89.34.10
  "url": "https://httpbin.org/get"
}
```

6. Verify the Response

From the output, identify:

1. **Response status code**
   Look at the first line (e.g. 200 OK).
2. **Response headers**
   Find headers like Content-Type and Content-Length.
3. **JSON body**
   The data returned by the API in JSON format.

Key Takeaway

- curl is a powerful tool for testing APIs.
- Use -i to see headers, -sS for cleaner output with errors.
- The response includes a status code, headers and a JSON body.

Common Beginner Mistakes

- Forgetting to include the full URL.
- Not checking the status code.
- Assuming every API returns JSON.
- Ignoring error messages when using -s without -S.
- Thinking a successful request always means the data is correct.

Sending JSON With POST Requests

VERIQTA

A POST request sends data to a server to create a new resource. We use JSON in the request body, and we must set the Content-Type header to `application/json`.

1. What Happens?

1. Client sends a POST request with JSON data.
2. Server receives and validates the data.
3. Server processes the request (e.g. create a task).
4. Server returns a response (e.g. 201 Created) with data in JSON format.

2. The Request Components

URL	The API endpoint (e.g. https://httpbin.org/post).
Method	POST (creates a new resource).
Headers	Content-Type: application/json tells the server the data is in JSON format.
Request Body	The JSON data you want to send (e.g. task details).

3. JSON Data to Send

Example JSON (new TaskFlow task):

```
{
  "title": "Learn DevOps",
  "status": "open",
  "priority": "high",
  "due_date": "2024-11-20",
  "tags": ["api", "practice"]
}
```

4. curl Command With -H and -d

Run this command in your terminal to send JSON to an echo endpoint:

```
curl -X POST https://httpbin.org/post \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Learn DevOps",
    "status": "open",
    "priority": "high",
    "due_date": "2024-11-20",
    "tags": ["api", "practice"]
  }'
```

5. Example Response (Truncated)

The server echoes back the data you sent.

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 390
{
  "json": {
    "title": "Learn DevOps",
    "status": "open",
    "priority": "high",
    "due_date": "2024-11-20",
    "tags": ["api", "practice"]
  },
  "url": "https://httpbin.org/post"
}
```

6. Explanation of Options

Option	What it Does
-X POST	Specifies the HTTP method (POST).
-H	Adds an HTTP header. Here we set Content-Type: application/json.
-d	Sends the request body data (JSON).
URL	The API endpoint to send the request to.

Common Failures

- Malformed JSON (missing quotes, commas, or brackets).
- Incorrect Content-Type (e.g. text/plain instead of application/json).
- Sending invalid data (wrong field names or data types).
- Forgetting the -d option (no body is sent).
- Using single quotes incorrectly (especially on Windows).

Key Takeaway

- Use Content-Type: application/json when sending JSON.
- The request body contains the data you want to send.
- Always check the response to confirm the server received and processed your data.

Headers, Tokens, and API Authentication

VERIQTA

APIs often require authentication to verify who you are and authorization to control what you can access. You send credentials (like an API key or bearer token) in headers with your request.

1. Key Concepts

Authentication

Proves who you are

(e.g. using an API key or token).

Authorization

Controls what you can access

(e.g. which resources or actions you are allowed to use).

2. How Authentication Works



Client

Sends a request with authentication header (API key or token).

Authorization Check

API server verifies the credentials and checks permissions.

Protected Resource

If valid, you get access to the requested resource.

3. Common Authentication Methods

Method	Header Example	When It's Used
API Key	X-API-Key: YOUR_API_KEY	Simple key provided by the API provider. Common in many public APIs.
Bearer Token	Authorization: Bearer YOUR_TOKEN	Used with OAuth 2.0 and many modern APIs. The token represents your identity and permissions.
Basic Authentication	Authorization: Basic base64(user:pass)	Uses a base64 encoded username and password. Less common and usually not recommended for modern APIs.

4. Example Requests With curl

Using an API Key

```
curl -i https://api.example.com/tasks \
  -H "X-API-Key: YOUR_API_KEY"
```

Using a Bearer Token

```
curl -i https://api.example.com/tasks \
  -H "Authorization: Bearer YOUR_TOKEN"
```

Multiple Headers (e.g. JSON + Token)

```
curl -i https://api.example.com/tasks \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json"
  -d '{"title": "Learn DevOps", "status": "open"}'
```

5. Protected and Forbidden Responses

Status Code	Meaning	Common Cause
401	Unauthorized	Missing or invalid authentication credentials (e.g. wrong or expired token).
403	Forbidden	You are authenticated but do not have permission to access the resource.

Security Reminder

- Never commit real API tokens to Git.
- Do not expose tokens in screenshots, logs, or public code.
- Use environment variables or a secrets manager to store tokens.
- Treat API tokens like passwords.

Common Beginner Mistakes

- Forgetting to include the authentication header.
- Using an expired or invalid token.
- Confusing authentication (who you are) with authorization (what you can do).
- Sharing API tokens in code, screenshots, or chat.

API Endpoints, Resources, and Versioning

VERIQTA

APIs expose resources through endpoints (URLs). REST-style APIs use clear, predictable URLs to represent resources. Versioning helps you introduce changes without breaking existing clients.

1. What Are Endpoints and Resources?

An endpoint is a URL where you send a request. A resource is a thing (e.g. a task) that the API manages.



2. Common Resource Endpoints (TaskFlow Example)

Action	HTTP Method	Endpoint	Description
List all tasks	GET	/api/v1/tasks	Get a list of tasks
Get a single task	GET	/api/v1/tasks/42	Get task with ID 42
Create a task	POST	/api/v1/tasks	Create a new task
Update a task	PUT	/api/v1/tasks/42	Replace the task
Partially update	PATCH	/api/v1/tasks/42	Update specific fields
Delete a task	DELETE	/api/v1/tasks/42	Delete the task

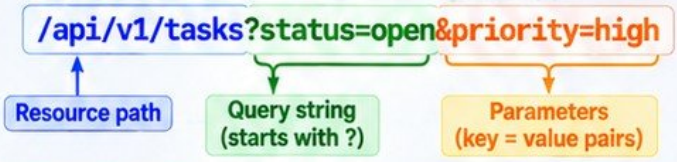
3. Path Parameters

Path parameters identify a specific resource in the URL.



4. Query Parameters (Optional)

Query parameters add extra options to a request.



5. API Versioning

Versioning helps you introduce changes without breaking existing clients.

/api/v1/tasks	Original version (still supported)
/api/v2/tasks	New version (with changes or new features)

6. Why Versioning Matters

- Allows you to add new features without breaking existing clients.
- Clients can migrate to a new version on their own timeline.
- You can make incompatible changes in a new version.
- Multiple versions can run at the same time.

7. Real Example: TaskFlow API Response (v1)

GET /api/v1/tasks/42

```
{
  "id": 42,
  "title": "Learn DevOps",
  "status": "open",
  "due_date": "2024-11-20"
}
```

8. Same Endpoint, Different Version (v2)

GET /api/v2/tasks/42

```
{
  "id": 42,
  "title": "Learn DevOps",
  "status": "open",
  "due_date": "2024-11-20",
  "assignee": "ann",
  "tags": ["api", "practice"]
}
```

Production Insight

Changing an API response field can break clients even when the endpoint remains available. Clients may rely on specific field names, data types, or formats. Always maintain backward compatibility or use a new version for breaking changes.

Common Beginner Mistakes

- Changing or removing fields in a response without versioning.
- Using unclear or inconsistent endpoint URLs.
- Not using versioning for breaking changes.
- Hardcoding API versions in a way that is difficult to update.

Best Practices

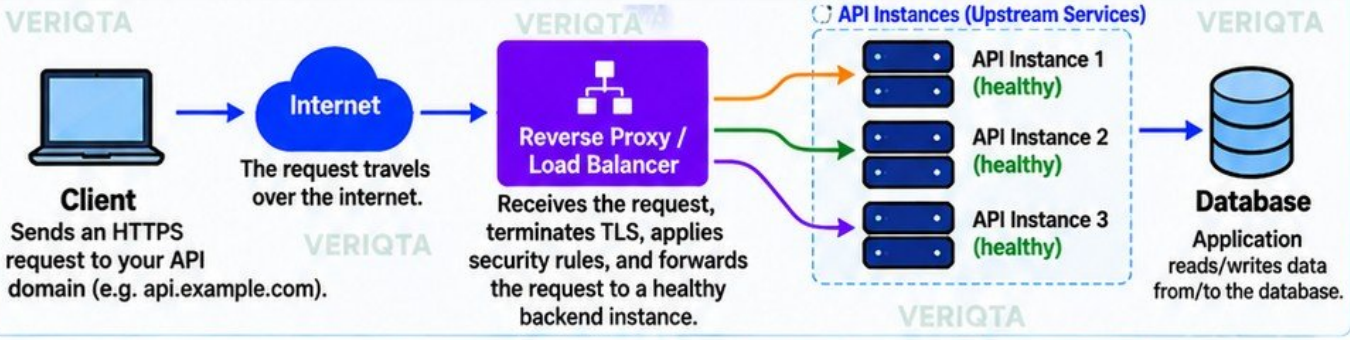
- Use clear, consistent, and descriptive URLs.
- Use versioning (e.g. /api/v1/).
- Keep responses backward compatible when possible.
- Document your API changes and version support.

APIs Behind Load Balancers and Reverse Proxies

VERIQTA

In production, API requests usually do not go directly to your application. They first go through a reverse proxy or load balancer, which handles security, traffic management, and routing to backend services.

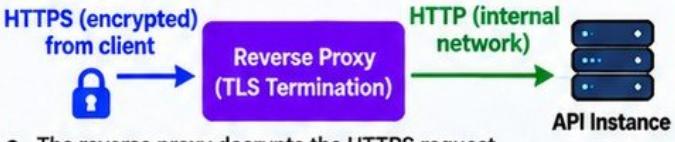
1. The Request Flow



2. What Each Component Does

Component	Role
Client	Sends a request to the API domain (e.g. https://api.example.com).
Reverse Proxy	Acts as a gateway. Can handle TLS termination, security, caching, and request routing.
Load Balancer	Distributes requests across multiple backend instances (for high availability).
TLS Termination	Decrypts HTTPS (TLS) at the proxy, then forwards the request to backend services (often over HTTP inside the network).
API Instances	Your application servers that handle the actual API requests.
Database	Stores and retrieves data for your API.

3. TLS Termination Explained



- The reverse proxy decrypts the HTTPS request.
- It reads the request and applies security rules (e.g. rate limiting, IP filtering).
- It forwards the request to backend services, usually over HTTP inside a private network.

4. Common Reverse Proxy / Load Balancer Tools

Nginx | HAProxy | Traefik | AWS Application Load Balancer | Kubernetes Ingress

5. Why It May Work Locally But Fail Through the Proxy

Works Locally (Direct Access)



- ✅ No TLS termination.
- ✅ No reverse proxy in between.
- ✅ Fewer security rules.
- ✅ Direct access to the application.
- ✅ May work even if some headers or configurations are missing.

Fails Through the Proxy



- ❌ Missing or incorrect headers (e.g. X-Forwarded-For).
- ❌ TLS/SSL or certificate issues.
- ❌ Path or URL rewriting by the proxy.
- ❌ Security rules blocking the request.
- ❌ Timeouts or health check failures.
- ❌ Different environment configuration (e.g. expecting HTTP instead of HTTPS).

Troubleshooting Tips

- ✅ Check the proxy and load balancer logs.
- ✅ Verify headers (e.g. X-Forwarded-For, Host).
- ✅ Confirm the correct protocol (HTTP vs HTTPS).
- ✅ Check firewall and security group rules.
- ✅ Ensure the backend service is healthy.
- ✅ Test the API both directly and through the proxy and compare the requests.

Key Takeaway

- Reverse proxies and load balancers add important security and reliability features.
- They can also introduce new points of failure.
- An API working locally does not guarantee it will work in production through the proxy.
- Always consider the full request path when troubleshooting.

API Health Checks: Is the Service Really Healthy?

VERIQTA

A health check is an endpoint used to check the status of an application. An HTTP 200 response only means the endpoint responded, but it does not always mean the entire application is healthy.

1. Types of Health Checks

Liveness Check



Checks if the application process is running.
If it fails, the process is restarted.
It does not check dependencies like the database.

Endpoint example:
`/health/live`

Readiness Check



Checks if the application is ready to serve traffic.
It verifies important dependencies (e.g. database, cache, external services).
If it fails, the service is removed from the load balancer but not restarted.

Endpoint example:
`/health/ready`

Dependency Check



Checks the health of external dependencies such as databases, message queues, or third-party services.

Endpoint example:
`/health/deps`

2. Example Health Endpoints (TaskFlow)

Liveness Check Response

```
GET /health/live                                   HTTP 200
{
  "status": "alive",
  "uptime": "2h 15m"
}
```

Readiness Check Response

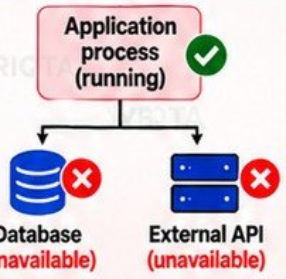
```
GET /health/ready                                  HTTP 200
{
  "status": "ready",
  "checks": {  "database": "ok",
               "cache": "ok"    }
}
```

Dependency Check Response

```
GET /health/deps                                   HTTP 200
{
  "database": "ok",
  "redis": "ok",
  "external_api": "ok"
}
```

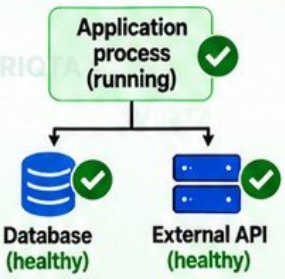
3. Healthy Process vs Ready-to-Serve Application

Process is Healthy But Not Ready



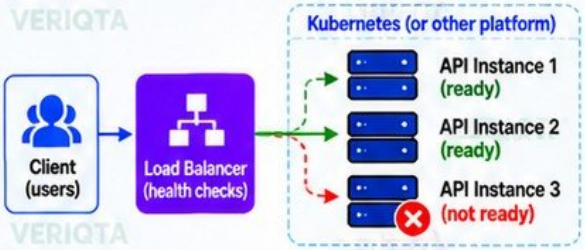
The process is running, so /health/live returns 200, but the app cannot serve requests because dependencies are down. /health/ready should return a non-200 status.

Fully Healthy and Ready



The process is running and all required dependencies are healthy. /health/live and /health/ready both return 200.

4. How It Works in Production



The load balancer or Kubernetes uses readiness checks to send traffic only to healthy and ready instances. Instances that fail the readiness check are removed from the load balancer.

5. Kubernetes Health Checks



Kubernetes uses probes to check the health of your application.

```yaml
livenessProbe:
  httpGet:
    path: /health/live
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /health/ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
```

6. Common Mistakes

- Using only a simple HTTP 200 check without verifying dependencies.
- Returning 200 even when the application is not fully ready.
- Not separating liveness and readiness endpoints.
- Including sensitive information in health responses.
- Forgetting to configure health checks in the load balancer or Kubernetes.

7. Key Takeaways

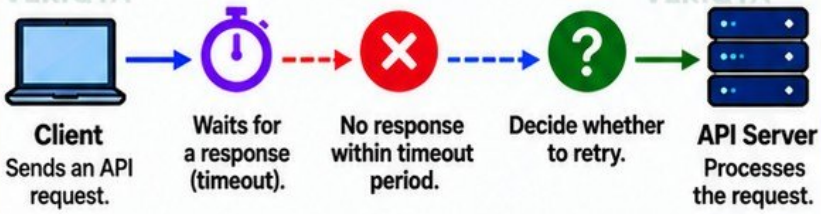
- A 200 response only means the endpoint responded.
- Use separate liveness and readiness checks.
- Check important dependencies (database, cache, external APIs).
- Kubernetes and load balancers use health checks to route traffic to healthy instances.
- Proper health checks improve reliability and reduce downtime.

API Timeouts, Retries, and Duplicate Requests

VERIQTA

Network delays and temporary failures can prevent an API request from completing. Clients use timeouts, retries, and backoff strategies to handle these issues. However, careless retries can create duplicate requests and make an outage worse.

1. The Request Flow



2. Types of Timeouts

Type	What It Means
Connection Timeout	Time to establish a connection with the server.
Response Timeout	Time to wait for the server to send a response.
Read Timeout	Time to wait while reading the response data.
Overall Timeout	Maximum time for the entire request to complete.

3. Retry Logic

If a request fails due to a temporary issue (e.g. network error or 5xx response), the client may retry the request.



4. Backoff Strategies

Backoff means waiting longer between retries to reduce pressure on the server.

Strategy	How It Works
Fixed Delay	Wait a constant time (e.g. 2 seconds) between retries.
Exponential Backoff	Wait longer each time (e.g. 1s, 2s, 4s, 8s).
Jitter	Add a random amount of time to avoid many clients retrying at the same time.

5. Idempotency and Duplicates

Idempotent Operations (safe to retry)

- **GET** — Read data (no changes).
- **PUT** — Replace data (same result).
- **DELETE** — Remove data (same result).

Non-Idempotent Operations (can create duplicates)

- **POST** — Create new resources (may create duplicates).
- **PATCH** — Partial updates (may have side effects).

Retrying POST requests without idempotency controls (e.g. idempotency keys) can create duplicate resources.

6. Example: Retry With curl

```
# Retry up to 3 times with 2s delay
for i in 1 2 3; do
  echo "Attempt $i..."
  response=$(curl -sS -w "%{http_code}"
  --max-time 5 https://api.example.com/tasks)
  if [ "$response" = "200" ]; then
    echo "Success!"
    break
  else
    echo "Request failed. Retrying in 2s..."
    sleep 2
  fi
done
```

7. Common Causes of Timeouts

- Network latency or congestion.
- Slow backend processing.
- Overloaded servers.
- Database or external service delays.
- Incorrect timeout configuration (too short).
- Firewall or proxy issues.

8. Production Warning

Uncontrolled retries can increase load during an outage.

- Many clients retrying at the same time can make the problem worse.
- It can cause higher traffic, longer response times, and complete service failure.
- Use retry limits, exponential backoff, and jitter.
- Monitor and alert on high retry rates.

9. Best Practices

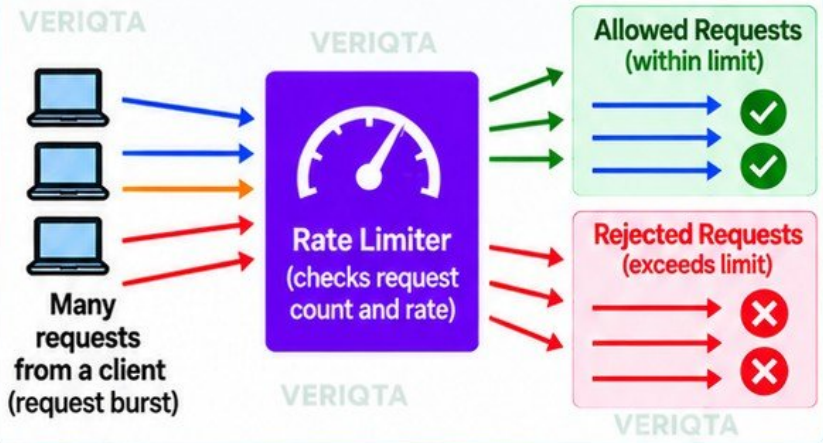
- Set appropriate timeout values.
- Use retry limits (e.g. 3 retries).
- Use exponential backoff with jitter.
- Understand which operations are idempotent.
- Monitor retry rates and error rates.
- Use idempotency keys for critical POST requests.

API Rate Limiting and HTTP 429

VERIQTA

Rate limiting controls how many requests a client can make within a specific time period. It helps APIs stay stable, fair, and available for all users. When you exceed the limit, the server returns HTTP 429 (Too Many Requests).

1. How Rate Limiting Works



2. Common Rate Limit Types

Type	Description
Requests per second (RPS)	Maximum number of requests allowed each second.
Requests per minute (RPM)	Maximum number of requests allowed each minute.
Requests per hour (RPH)	Maximum number of requests allowed each hour.
Per user / API key	Limit applied to a specific user or API key.
Per IP address	Limit applied to a client's IP address.

3. Example: HTTP 429 Response

```
HTTP/1.1 429 Too Many Requests
Date: Wed, 20 Nov 2024 10:00:00 GMT
Content-Type: application/json
Retry-After: 30
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1700473200

{
  "error": "rate_limit_exceeded",
  "message": "You have made too many requests.",
  "retry_after": 30
}
```

4. Important Response Headers

Header	Meaning
Retry-After	Tells you how long to wait before making another request (in seconds or a date/time).
X-RateLimit-Limit	Total number of requests allowed in the time window.
X-RateLimit-Remaining	Number of requests remaining in the current window.
X-RateLimit-Reset	Time when the rate limit window resets (Unix timestamp).

5. Why APIs Use Rate Limiting

- Protects against abuse and bots.
- Prevents server overload.
- Ensures fair usage for all users.
- Maintains stability and availability.
- Helps control costs and resource usage.

6. Handling 429 Responses

1. Check the Retry-After header and wait.
2. Use exponential backoff for retries.
3. Reduce the request rate.
4. Implement caching to avoid unnecessary requests.
5. Monitor your usage and stay within the limits.

7. Example: Retry Logic With curl

```
# Simple example: wait and retry after 429
for i in 1 2 3; do
  response=$(curl -s -o /dev/null -w "%{http_code}" \
    https://api.example.com/tasks)
  if [ "$response" = "429" ]; then
    echo "Rate limited. Waiting 30 seconds..."
    sleep 30
  else
    echo "Request successful (HTTP $response)"
    break
  fi
done
```

8. Key Takeaways

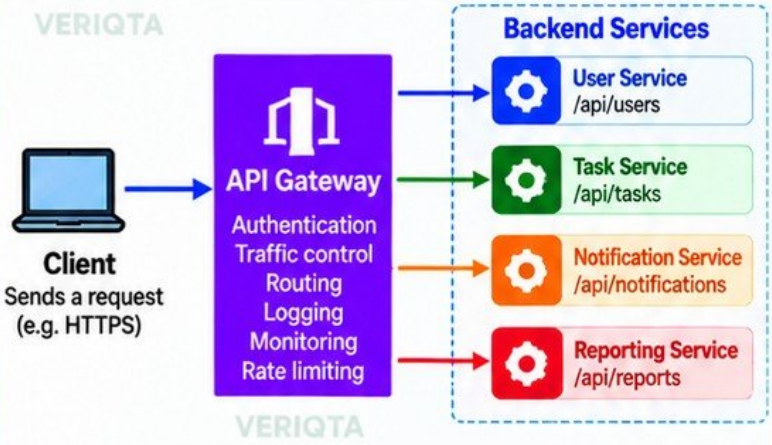
- HTTP 429 means you have exceeded the rate limit.
- A 429 response is not necessarily an application outage.
- Always check the Retry-After header.
- Use backoff, caching, and proper request limits.
- Design your clients to handle rate limits gracefully.

API Gateways: The Front Door to Services

VERIQTA

An API gateway sits in front of your backend services. It receives client requests, applies policies such as authentication, rate limiting, and traffic control, and then routes the requests to the appropriate service.

1. How an API Gateway Works



2. Key Functions of an API Gateway

- **Authentication** — Validates API keys, tokens, or other credentials.
- **Authorization & Policy Enforcement** — Enforces access rules and security policies.
- **Request Routing** — Routes requests to the correct backend service.
- **Rate Limiting** — Controls the number of requests from a client.
- **Load Balancing** — Distributes traffic across multiple instances.
- **Caching** — Can cache responses to improve performance.
- **Logging & Monitoring** — Records requests, responses, and metrics.
- **Traffic Control** — Can block, transform, or throttle requests.

3. Example: Routing Requests



4. Real-World API Gateway Examples

AWS API Gateway · Kong · NGINX (plus) · Traefik · Azure API Management

5. Example: Request Flow With Policies



6. Common Beginner Mistakes

- Assuming the API gateway replaces the application.
- Thinking the gateway handles all authorization checks.
- Not configuring proper rate limits.
- Exposing internal services directly instead of using the gateway.
- Ignoring logging and monitoring.
- Believing the gateway solves all security issues by itself.

7. Benefits of Using an API Gateway

- Provides a single entry point for multiple services.
- Centralizes authentication and security policies.
- Simplifies client integration.
- Helps control and manage traffic.
- Improves observability with centralized logging and metrics.
- Can offload common tasks (rate limiting, caching, encryption).

8. Important Note

An API gateway does not replace your application or its authorization checks.

Your services should still validate user permissions and enforce business rules, even when requests come through an API gateway.

9. Key Takeaways

- An API gateway is the front door to your services.
- It can authenticate, route, and control traffic.
- It improves security, reliability, and observability.
- It does not replace your application or its internal authorization checks.
- Use an API gateway as part of a broader security and architecture strategy.

API Testing Before Deployment

VERIQTA

Testing your API before deployment helps catch issues early, prevents breaking changes, and ensures your API behaves as expected. Use positive tests, negative tests, contract checks, and automated tests in your CI/CD pipeline.

1. Types of API Tests

Positive Tests

Verify that the API returns the expected status code, headers, and JSON fields for valid requests.

Example:
GET /api/tasks
→ 200 OK

Negative Tests

Verify that the API handles invalid requests correctly (e.g. missing fields, invalid data, unauthorized access).

Example:
GET /api/tasks/999
→ 404 Not Found

Contract Checks

Ensure the API response matches the expected structure (required fields, data types, etc.). Helps detect breaking changes.

Example:
Response must include "id", "title", "status"

CI/CD Pipeline Validation

Run automated API tests in your pipeline. Fail the deployment if critical tests fail.

Example:
Block deployment if status code or required fields are missing.

2. Basic API Test Flow

Send API Request (positive or negative)
↓
Check Status Code (e.g. 200, 400, 404)
↓
Validate Response Body (required JSON fields)
↓
Pass or Fail Test (in CI/CD pipeline)

3. Practical: Test an API With curl

```
# 1. Positive test: check status code and JSON fields
curl -sS https://httpbin.org/json | jq .

# Check status code only
curl -s -o /dev/null -w "%{http_code}\n" \
    https://httpbin.org/json

# 2. Negative test: invalid endpoint
curl -s -o /dev/null -w "%{http_code}\n" \
    https://httpbin.org/invalid-endpoint

# 3. Test for required JSON field (using jq)
curl -sS https://httpbin.org/json | jq -e '.slideshow'
# Returns the field if it exists, or an error if missing
```

4. Example: API Test in CI/CD Pipeline (GitHub Actions)

```
name: API Tests
on: [push]
jobs:
  test-api:
    runs-on: ubuntu-latest
    steps:
      - name: Run API tests
        run: |
          # Check status code is 200
          status=$(curl -s -o /dev/null -w "%{http_code}" \
            https://httpbin.org/json)
          if [ "$status" != "200" ]; then
            echo "API test failed: expected 200, got $status"
            exit 1
          fi
          # Check required JSON field
          curl -sS https://httpbin.org/json | jq -e '.slideshow'
```

5. Common API Test Scenarios

Scenario	Example	Expected Result
Valid request (GET)	GET /api/tasks	200 and correct JSON fields
Invalid ID	GET /api/tasks/999	404 Not Found
Missing field (POST)	POST /api/tasks { "title": "" }	400 Bad Request
Unauthorized	GET /api/tasks (without token)	401 Unauthorized
Invalid token	GET /api/tasks (with bad token)	403 Forbidden
Rate limit exceeded	Multiple requests	429 Too Many Requests

6. Why API Testing Matters

- Catches breaking changes before production.
- Ensures the API meets expected behavior.
- Reduces incidents and downtime.
- Automates quality checks in CI/CD.
- Builds confidence when deploying new versions.

7. Beginner Mistakes

- Only testing happy paths (positive tests).
- Not checking response body fields.
- Ignoring authentication and error scenarios.
- Not running API tests in the CI/CD pipeline.
- Assuming a successful status code always means the API is correct.

8. Key Takeaways

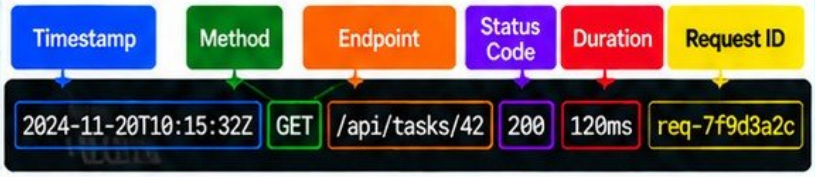
- Test both positive and negative scenarios.
- Verify status codes and required JSON fields.
- Use contract checks to detect breaking changes.
- Automate API tests in your CI/CD pipeline and fail the deployment when tests fail.
- API testing improves reliability and reduces production issues.

Reading API Logs and Request IDs

VERIQTA

API logs record details about each request, including timestamps, HTTP methods, response codes, request duration, and correlation or request IDs. These logs help you troubleshoot issues, understand system behavior, and trace a request across multiple services.

1. Anatomy of an API Log Entry



Timestamp	When the request was received (usually in UTC).
HTTP Method	The action performed (GET, POST, PUT, DELETE, etc).
Endpoint	The resource that was requested.
Response Code	The result of the request (e.g. 200, 404, 500).
Duration	How long the request took to process.
Request ID	A unique identifier used to trace this request across services.

2. From Failed Request to Server Log

1. Client makes a request and gets an error



HTTP 500 Internal Server Error

Request ID: req-7f9d3a2c

2. Search for the same request ID in the server logs

Server Log

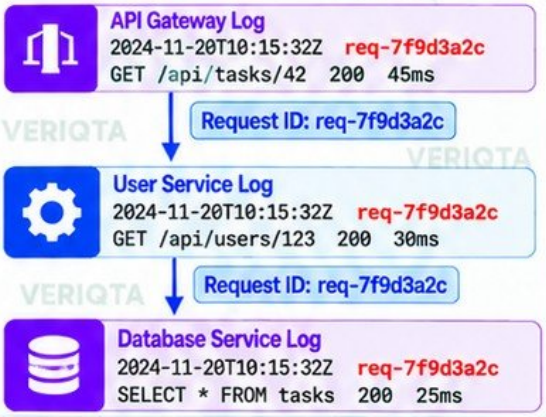
```
2024-11-20T10:15:32Z ERROR req-7f9d3a2c
GET /api/tasks/42 user_id=123
duration=120ms status=500
message="Database connection failed"
```

3. Common HTTP Methods and Response Codes in Logs

Method	Meaning
GET	Retrieve data.
POST	Create new data.
PUT	Update existing data (replace).
PATCH	Update part of the data.
DELETE	Remove data.

Status Code	Meaning
200	Success (OK).
201	Created.
400	Bad Request (client error).
401	Unauthorized.
403	Forbidden.
404	Not Found.
429	Too Many Requests.
500	Internal Server Error (server error).

4. Example: Tracing a Request Across Services



5. Practical: Find a Request in Logs

```
# Search for a specific request ID in logs
grep "req-7f9d3a2c" /var/log/api.log

# Search for errors related to a request ID
grep "req-7f9d3a2c" /var/log/api.log | grep ERROR

# View logs in real time and filter by request ID
tail -f /var/log/api.log | grep "req-7f9d3a2c"
```

6. Security Reminder

- Do not log passwords.
- Do not log API tokens or keys.
- Avoid logging sensitive personal data (e.g. emails, phone numbers, addresses).
- Mask or redact sensitive information in logs.
- Follow your organization's security and privacy policies.

7. Key Takeaways

- Logs show timestamps, methods, status codes, duration, and request IDs.
- Use request IDs to trace a request across multiple services.
- Correlate client errors with server logs.
- Logs are essential for troubleshooting and monitoring.
- Always protect sensitive data in logs.

API Monitoring: Latency, Traffic, Errors, Saturation

Monitoring helps you understand how your API is performing in real time. Engineers track request rates, response times (latency), error rates, and resource saturation to detect issues early and keep the service reliable for users.

1. Key Metrics to Monitor

Request Rate (Traffic)

Number of requests received per second (RPS). Shows how much the API is being used.

Response Time (Latency)

Time taken to process a request and return a response. Often measured using p50, p95, and p99.

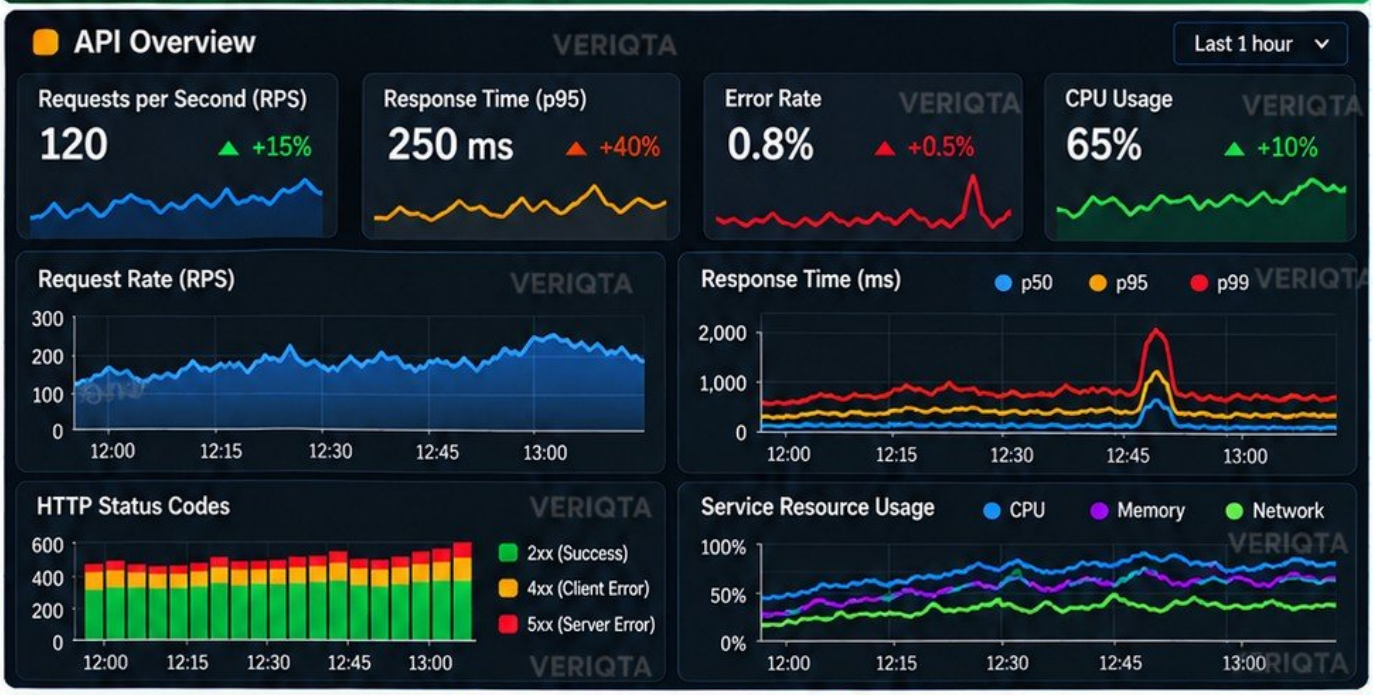
Error Rate

Percentage of requests that fail (e.g. 4xx or 5xx responses). High error rates indicate problems.

Resource Saturation

CPU, memory, network, and database usage. Shows if the service is reaching its limits.

2. Example API Monitoring Dashboard



3. Why API Monitoring Matters

- Detect issues before users report them.
- Understand usage patterns and plan capacity.
- Identify unusual activity (e.g. spikes, abuse).
- Improve reliability and performance.

4. Production Insight

An API can return HTTP 200 (success) while still being too slow for users.

Even if the status code is 200, high response times (e.g. p95 latency) can make the service unusable. Always monitor latency and user experience, not just status codes.

5. Common Alerts

- High response time (p95 or p99).
- Increased error rate (4xx or 5xx).
- Unusual spike in traffic.
- High CPU or memory usage.
- Database or external service latency.

6. Key Takeaways

- Monitor request rate, latency, error rate, and resources.
- Set up alerts for unusual changes.
- Use dashboards to understand trends.
- A 200 response does not always mean the API is healthy.
- Monitoring helps you keep your API reliable and users happy.

Production Incident: The API Returns 503

VERIQTA

In this real-world scenario, the TaskFlow API starts returning HTTP 503 (Service Unavailable) after a deployment. You will learn how to investigate the issue by checking symptoms, readiness, upstream connections, logs, and recent changes, and decide when a rollback is the safer option.

1. Incident Scenario

After a new deployment, users report that TaskFlow is returning **HTTP 503** errors. Some requests work, but most are failing. You are on call and need to investigate.

Symptoms

- Users see: 503 Service Unavailable
- Some endpoints work, others fail
- Increased error rate in monitoring
- Started right after a new deployment

2. Example Error Response

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/json
Retry-After: 30

{
  "error": "service_unavailable",
  "message": "TaskFlow API is temporarily
   unavailable.",
  "request_id": "req-8f9d3a2c"
}
```

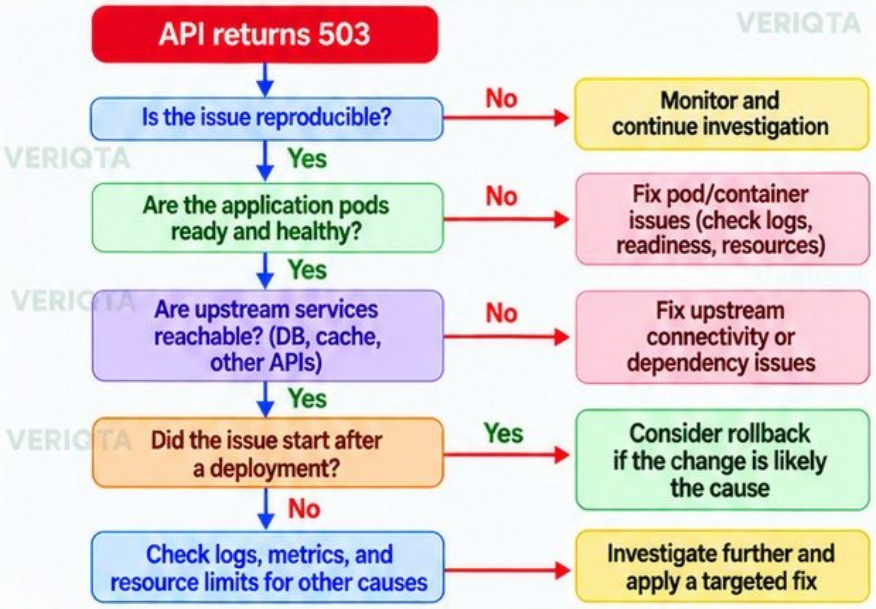
3. Check Recent Changes

- Review the deployment that triggered the incident.
- Check what code changes were included.
- Look for infrastructure changes (e.g. config, env vars, secrets).
- Verify if any dependencies were updated.
- Check the exact time the issue started.

4. Investigation Steps

1. **Confirm the issue**
   Reproduce the 503 error and check which endpoints are affected.
2. **Check service health**
   Verify pod/container readiness and health checks.
3. **Inspect upstream services**
   Check if databases, caches, or external APIs are reachable.
4. **Look at logs**
   Search for errors using the request ID and timestamp.
5. **Check resource usage**
   Look for high CPU, memory, or connection limits.
6. **Compare with previous version**
   Identify what changed in the new deployment.

5. Troubleshooting Decision Flow



6. Useful Commands

```
# Check pod status
kubectl get pods -l app=taskflow

# View pod logs
kubectl logs <pod-name> --tail=100

# Check recent deployments
kubectl rollout history deployment/taskflow

# Check service and endpoints
kubectl get svc taskflow
kubectl get endpoints taskflow
```

7. When to Rollback?

Rollback is safer when:

- The issue started immediately after a deployment.
- The new change is likely the cause.
- Users are significantly impacted.
- A quick rollback can restore service.
- Further investigation will take too long and the service needs to be available.

8. Key Takeaways

- Investigate systematically (symptoms, health, upstream, logs).
- Use request IDs to find relevant logs.
- Check recent changes and compare with previous version.
- Monitor metrics to identify the root cause.
- Rollback is a valid and safe option when the incident is caused by a recent change and users are impacted.

Junior Engineer API Challenge

VERIQTA

A short practical incident to test your API troubleshooting skills. Follow the steps, analyze the output, answer the questions, and propose a fix. This challenge simulates a real TaskFlow API issue.

1. Incident Scenario

After a new deployment, the TaskFlow API is failing for some users. The web app shows an error when trying to create a new task. Your job is to investigate the issue and restore normal operation.

2. API Request (Failing)

```
$ curl -i -X POST \
  https://api.taskflow.local/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Test task","user_id":123}'

HTTP/1.1 503 Service Unavailable
Date: Mon, 20 Oct 2026 14:32:10 GMT
Content-Type: application/json
Retry-After: 30
X-Request-ID: req-9f8e7d2a1c

{
  "error": "service_unavailable",
  "message": "Database connection failed",
  "request_id": "req-9f8e7d2a1c"
}
```

3. Questions

1. What is the HTTP status code returned by the API?
2. What does the Retry-After header indicate?
3. Use the response data. What is the main error message?
4. Which command would you run next to check if the TaskFlow API is reachable at the network level?
5. Based on the information, what is a likely cause of the issue?
6. What fix would you propose?
7. How would you verify that the API has recovered?

4. Check the Headers

HTTP Status	503 Service Unavailable
Content-Type	application/json
Retry-After	30 (seconds)
X-Request-ID	req-9f8e7d2a1c

5. Inspect the Response Data

```
{
  "error": "service_unavailable",
  "message": "Database connection failed",
  "request_id": "req-9f8e7d2a1c"
}
```

6. Choose the Next Diagnostic Command

- **A** `ping api.taskflow.local` — Check basic network connectivity.
- **B** `curl -I https://api.taskflow.local` — Check if the API endpoint responds.
- **C** `kubectl get pods -n taskflow` — Check the status of TaskFlow pods.
- **D** `kubectl logs deployment/taskflow-api` — View API logs for more details.

7. Investigate the Root Cause

```
$ kubectl logs deployment/taskflow-api
2026-10-20T14:32:08Z  ERROR  Failed to
connect to database: dial tcp
10.0.12.34:5432: connection refused
2026-10-20T14:32:09Z  WARN  Retrying
database connection...
2026-10-20T14:32:10Z  ERROR  Could not
establish database connection
```

8. Apply a Fix and Verify

1. **Fix the issue**
   Example: Restart the database, fix the database configuration, or roll back the deployment if the new version is the cause.
2. **Verify the API**
   Run the request again and confirm the expected response.
3. **Check the response**
   You should see HTTP 200 and the new task details in JSON.
4. **Confirm service health**
   Check logs and monitoring to ensure the service is stable.

Expected Successful Response

```
$ curl -i -X POST \
https://api.taskflow.local/api/tasks \
-H "Content-Type: application/json" \
-d '{"title":"Test task","user_id":123}'

HTTP/1.1 200 OK
Content-Type: application/json
X-Request-ID: req-9f8e7d2a1c

{
  "id": 456,
  "title": "Test task",
  "user_id": 123,
  "status": "created"
}
```

9. Key Takeaways

- Always check the HTTP status code, headers, and response data.
- Use the request ID to find relevant logs.
- Check service health (pods, dependencies, databases).
- Verify recent changes (deployments, configuration).
- Apply a fix and confirm the API is working again.
- Follow the cycle: **Observe → Diagnose → Fix → Verify.**

10. Real-World Tips

- A 503 error often means a temporary issue, such as an unavailable dependency.
- The **Retry-After** header tells you when to try again.
- Check upstream services (databases, caches, external APIs).
- Look at logs, metrics, and recent deployments.
- If the issue started after a deployment and users are impacted, consider a rollback.
- Always verify the fix and monitor the service after recovery.